A Comparison of Robotic Path-Planning Algorithms

Sean Kent Jack Bernatchez S. Violet Killy

***Abstract*—One critical computational problem in autonomous mobile robotics is the challenge of path planning, in which a robot must navigate from point A to point B in an environment while avoiding collisions with obstacles. In recent years, many sampling based algorithms have been introduced for path-finding, with one of the most popular options being Rapidly-exploring Random Trees (RRT). While RRT does not guarantee an optimal solution, it is known for being quick, efficient, and exhibiting probabilistic completeness, making it useful for complex, high-dimensional problems. One extension of this algorithm, RRT*, achieves convergence towards an optimal solution, albeit with a slow convergence rate. We have also put forth our own extensions to the algorithm, referred to in this paper as RRT-march and RRT*-march. Probabilistic RoadMap (PRM) is an alternative to RRT in the same family of algorithms, one which adopts a different method for roadmap generation. PRM, like RRT, has an extension, PRM*, that is asymptotically optimal. We use simulation-based experiments to compare the performance of these algorithms in cluttered environments, comparing across metrics of run time and path cost.**

***Index Terms*—RRT, RRT*, PRM, PRM*, path planning, mobile robots**

I. Introduction

Motion planning (or path planning) is the task of finding a feasible trajectory from a starting point to a goal point, which avoids colliding with obstacles in the environment. The problem of path planning for mobile robots has exciting implications for a number of fields including autonomous vehicles, space exploration, industrial operations, and surveillance missions.

There are many subsets of effective algorithms for finding solutions to the path-planning problem, including "geometric, grid-based, potential field, neural networks, genetic, and sampling based" [4]. In this paper we focus on the sampling based family of methods, which are among the most popular and most recent, and have the advantage of being less computationally complex. Of these algorithms, Rapidly Exploring Random Tree (RRT) and Probabilistic RoadMap (PRM) are arguably the most influential.

The quickest algorithm to find an initial path, RRT was first conceptualized by LaValle in 1998 [2], as a path planning tool for high-dimensional, complex spaces. The algorithm constructs a randomized, space-filling tree of nodes in a given configuration space. It iteratively generates a random sample in the space, checks that it is in an obstacle-free region, and connects the sample to the nearest feasible node in the tree. The process continues until a path is found or a maximum number of iterations is reached [5].

An example of a resulting tree is shown in Figure 1. The obvious setback of the basic RRT algorithm is that it does not guarantee asymptotic optimality [4]. To address this, many extensions to the RRT algorithm have been proposed, with the largest breakthrough being the introduction of Rapidly Exploring Random Tree Star (RRT*).

Fig. 1. An example of a tree generated by the RRT family of algorithms. [2]

RRT* inherits the features of RRT and has a similar methodology. Unlike its predecessor, however, RRT* improves its path-finding by finding the best parent node for a new sample before connecting it to the tree. RRT* also introduces a cost-minimizing element to the algorithm, resulting in a smoother and more optimal path than RRT. Specifically, each time a new node is added to the network, neighboring nodes are rerouted through the new node if it makes their total cost (length of the path to the start node) lower. This results in seemingly less random trees, usually with many branches coming from the same nodes, creating fan-like structures throughout the tree, which results in straighter paths consisting of fewer nodes. On the downside, it involves substantially more computation which means that it can take a significantly longer time to find a path.

In addition to probabilistic completeness, RRT* converges toward an optimal solution, though it never reaches one in finite time due to slow convergence rates.

In this paper, we also explore the version of the algorithm presented in 6.881, which we dubbed RRT-March (and RRT*-March). The key difference with RRT-March is that after a new node is added to the tree, additional nodes are added in the same direction until either a collision occurs, or we reach the original sample point. This results in long, straight branches throughout the tree that would not occur from randomly sampled points alone. Essentially, each time a new node is added, the branch "marches" forward as far as it can, hence the name RRT-March. RRT*-March is simply RRT* with this same modification. This modification is intended to make the tree form more simple branches, and more efficient paths as opposed to regular RRT.

PRM is an alternative (though also probabilistically complete) sampling based planning algorithm, consisting of two phases: a construction phase and a query phase [3]. The techniques employed in the construction phase are what differentiates PRM from RRT, as it is aimed more at multiple-query applications. PRM starts with a random configuration, which is then connected with some number of neighbors in a given radius. Configurations are continuously added, until the roadmap is sufficiently dense. In the query phase, the shortest path from start to goal node is found within the dense roadmap. As with RRT, there is an extension to the PRM algorithm, PRM*. In this extension, the connection radius in the construction phase is scaled as a function of the number of samples.

II. Related Works

In recent years there have been extensive studies conducting comparative analysis of sampling based path planning algorithms [1]. A study by Noreen, Khan, and Haviv compared the RRT algorithm against two extensions, RRT* and RRT*-Smart [5] . The latter is a variation of RRT that introduces a path optimization process in order to accelerate the convergence towards an optimal solution, and to further cut down on path cost. The study compared performance in various environments across metrics of average path cost, average run time, and average tree density. The path cost analysis predictably found that RRT, RRT*, and RRT*-Smart saw decreasing costs (in that order). RRT* saw significantly higher run times than the other two variations, with RRT expectedly being the fastest algorithm. The path optimization and intelligent sampling techniques employed by RRT*-Smart resulted in a significantly less dense tree.

Another study compared performance of both RRT and PRM variants, and confirmed that both RRT* and PRM* converged to optimal solutions [3]. In addition, the researchers found that the path cost of PRM* and RRT* were significantly lower than their basic counterparts. Overall, it was shown that RRT* has the advantage over other algorithms in both time and space complexity, defined as the memory and time required for execution, respectively.

III. Results

To compare these algorithms, we ran repeated trials of each algorithm on randomly-generated environments. Each environment was of the same size, but consisted of randomly placed obstacles. For each randomly generated environment, we ran each algorithm, so the test-space is the same for each algorithm. The goal of this process was to compare the execution times, as well as the path-efficiency of each algorithm.

As related to path efficiency (see Figure 2), RRT* was the champion among the algorithms, with RRT*-March as a close second. While all of the algorithms were able to produce a minimum-bin path at least once, RRT* was able to do it very consistently. PRM, on the other hand, was the least consistent of the algorithms, producing paths ranging from size 20 all the way to size 75. Basic RRT and RRT-March came out in the middle of the pack, with RRT-March producing substantially more minimum-bin paths than RRT.

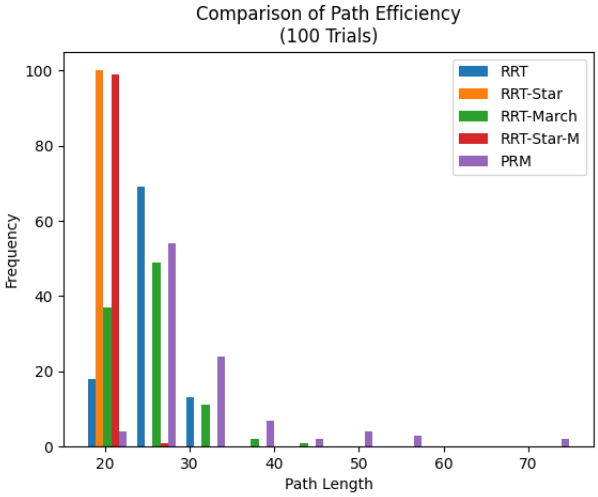

Fig. 2. Comparing planning algorithms by path efficiency .

The other metric tested was execution time (see Figure 3). Execution time is very important for a robot to be able to quickly react to its environment and update its plans in real-time. Our results showed that among these algorithms, RRT* was much less efficient than the other algorithms. Interestingly, RRT*-March performed substantially better than RRT* in these trials.

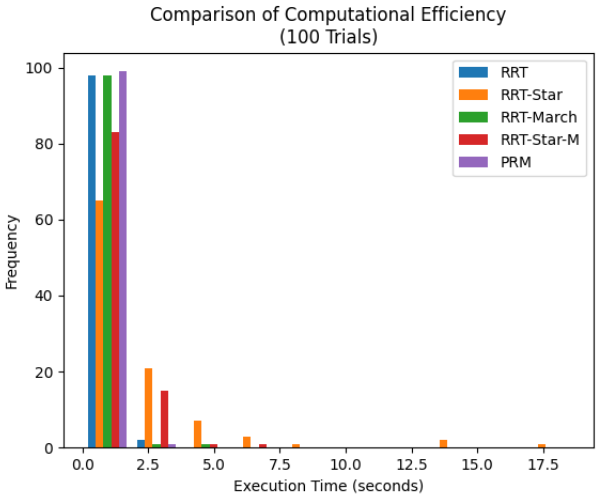

Fig. 3. Comparing planning algorithms by computational efficiency .

After removing outliers caused by RRT*'s poor performance, we were able to look more closely into the performance of the other algorithms (see Figure 4). Our results showed that RRT-March was the champion among these algorithms when it came to execution time, by a pretty large margin. RRT and RRT*-March did fairly well, while PRM, although consistent, tended to lag behind them by about

a quarter of a second. Additionally RRT* did occasionally perform at the highest level, but it is hard to discount the extreme outliers that it resulted in.

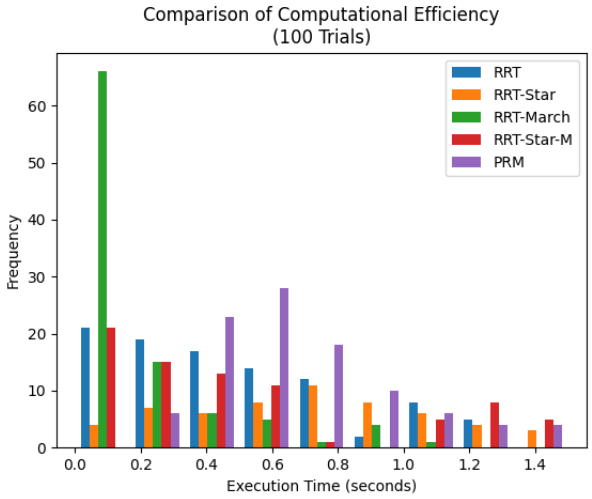

Fig. 4. Computational efficiency comparison with outliers removed.

In the previous tests, we performed one search per algorithm on each environment. This is to simulate real-life scenarios where the environment is rapidly changing. In the next tests, we wanted to compare RRT and PRM in a static environment. For each trial, instead of updating the locations of the obstacles, we simply moved the start and goal nodes around the environment. We predicted that PRM would have an advantage here, since it would be able to produce it's roadmap just once, and then simply perform A* search on each new start/goal location.

When it came to computational efficiency, PRM greatly outperformed RRT in this case, as we predicted (see Figure 5). This result makes a lot of sense, since PRM can find a path on each test with a quick search, while RRT had to construct a new tree on each new test.

As for path efficiency, RRT slightly outperformed PRM (see Figure 6). The distributions are fairly similar, but occasionally PRM resulted in some pretty inefficient paths, while RRT was much more consistent.

IV. Discussion

Our simulations show that each of these algorithms are useful in various situations. If you are in a rapidly changing environment and need to update your plans in real-time, then it seems that RRT-March is the way to go. If you have a little bit of extra time to spare, and are worried about path efficiency, then RRT*-March is a great choice. If your environment is static, and you are performing multiple tasks consecutively, then PRM (or PRM*) could be a good choice. If checking collisions is very expensive, then it may be a bad idea to use RRT*, as it will involve much more collision checking. On the other hand if checking collisions is cheap, then you will see less of a performance hit from using RRT*. The key insight

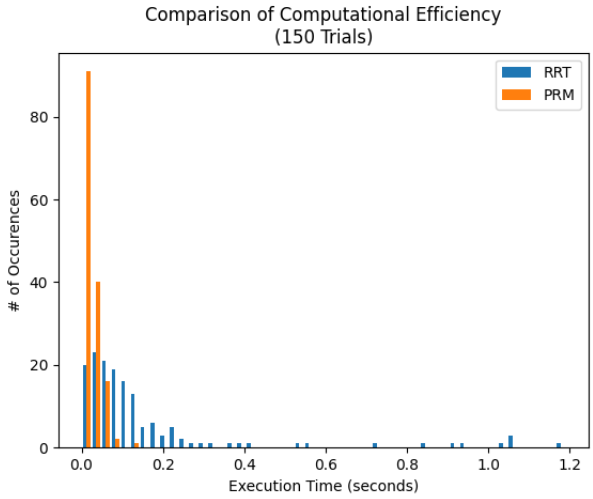

Fig. 5. Computational efficiency of PRM vs RRT.

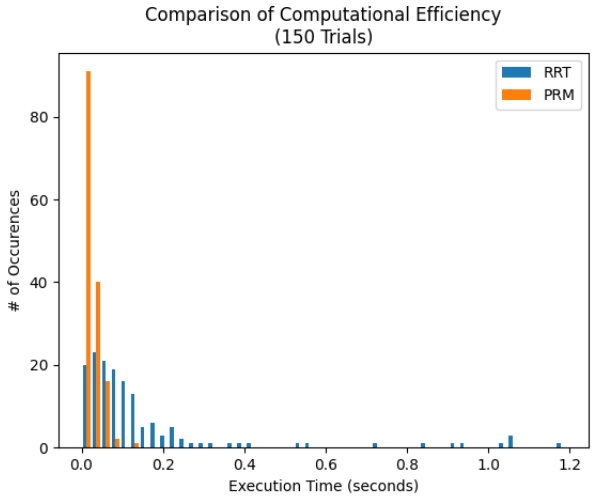

Fig. 6. Path efficiency of PRM vs RRT.

is that every problem is different, and things that work well in some environments may not work well in others. However, there has to be a champion; with near-perfect path efficiency, and an impressive improvement on time efficiency relative to RRT*, our number one motion planning algorithm based on our trials is RRT*-March.

Algorithm Power Rankings:

RRT*-March
RRT-March
RRT*
PRM
RRT

Our future plans for this project include exploring other motion planning algorithms and comparing them against similar

metrics. In addition to collision detection, we would like to implement other constraints such as joint configurations and torque limits. Also, now that we are confident in our algorithm functionality, we would like to implement our algorithms in Drake on the Kuka iiwa and view them as real-time simulations.

REFERENCES

[1] Geraerts, Roland (2006) "Sampling-based Motion Planning: Analysis and Path Quality", Ph.D. thesis. Utrecht University.

[2] LaValle, S.M. *Rapidly-Exploring Random Trees: A New Tool for Path Planning*; Technical Report; Computer Science Dept., Iowa State University: Ames, IA, USA, 1998.

[3] Karaman, S. and Frazzoli, E. (2011) "Sampling-based Algorithms for Optimal Motion Planning", International Journal of Robotics Research, vol. 30, no. 7, pp. 846–894.

[4] Nasir, J.; Islam, F.; Malik, U.; Ayaz, Y.; Hasan, O.; Khan, M.; Saeed, M. RRT*-SMART: A Rapid Convergence Implementation of RRT*. *Int. J. Adv. Robot. Syst.* 2013, 10, 1–12.

[5] Noreen, I.; Khan, A.; Habib, Z. A Comparison of RRT, RRT* and RRT*-Smart Path Planning Algorithms. *Int. J. Comput. Sci. Netw. Secur.* 2016, 16, 20–27.